Visitor Pattern in C++

Alright, let's take it to the next level! 🐾

We'll add:

- Another **Visitor**: `Zookeeper`
- Reuse our `Dog` and `Cat` classes
- Show how **adding new operations** (like `feed`, `clean`, etc.) is easy with the Visitor Pattern — **without modifying** `Dog` or `Cat`

🧩 Updated Code with Multiple Visitors

cpp

```
`#include <iostream>
#include <memory>
#include <vector>
using namespace std;

// Forward declarations
class Dog;
class Cat;

// Visitor Interface
class AnimalVisitor {
public:
    virtual void visit(Dog& dog) = 0;
    virtual void visit(Cat& cat) = 0;
    virtual ~AnimalVisitor() {}
};

// Animal Interface
class Animal {
public:
    virtual void accept(AnimalVisitor& visitor) = 0;
    virtual ~Animal() {}
};

// Concrete Animal: Dog
class Dog : public Animal {
public:
    void accept(AnimalVisitor& visitor) override {
        visitor.visit(*this);
    }
};

// Concrete Animal: Cat
class Cat : public Animal {
public:
    void accept(AnimalVisitor& visitor) override {
        visitor.visit(*this);
    }
};

// Concrete Visitor: Vet
class Vet : public AnimalVisitor {
public:
    void visit(Dog& dog) override {
        cout << "Vet gives the dog a vaccine and a treat." << endl;
    }

    void visit(Cat& cat) override {
```

```
        cout << "Vet gives the cat a check-up and gets hissed at." << endl;
    }
};

// Concrete Visitor: Zookeeper
class Zookeeper : public AnimalVisitor {
public:
    void visit(Dog& dog) override {
        cout << "Zookeeper feeds the dog and plays fetch." << endl;
    }

    void visit(Cat& cat) override {
        cout << "Zookeeper cleans the cat's litter box and gets ignored." << endl;
    }
};

int main() {
    vector<unique_ptr<Animal>> animals;
    animals.push_back(make_unique<Dog>());
    animals.push_back(make_unique<Cat>());

    Vet vet;
    Zookeeper keeper;

    cout << "--- Vet Visit ---" << endl;
    for (auto& animal : animals) {
        animal->accept(vet);
    }

    cout << "\n--- Zookeeper Visit ---" << endl;
    for (auto& animal : animals) {
        animal->accept(keeper);
    }

    return 0;
}
```

✅ Output:

```pgsql
--- Vet Visit ---
Vet gives the dog a vaccine and a treat.
Vet gives the cat a check-up and gets hissed at.

--- Zookeeper Visit ---
Zookeeper feeds the dog and plays fetch.
Zookeeper cleans the cat's litter box and gets ignored.
```

✨ Why This Is Powerful

Action	Change Required
Add a new **animal** (e.g., `Bird`)	Add new class and update **each visitor**
Add a new **operation** (e.g., `Trainer`, `Photographer`)	Just add a new visitor — **no changes to animals** ✔️

It's **open for extension**, **closed for modification** — straight from the **Open/Closed Principle** of SOLID design.

Want to go even further? We can add a `Bird`, or a `Trainer` visitor that teaches tricks. Or we can diagram this in UML. Just say the word. 😄